COS710 - Artificial Intelligence 1: Assignment 3

Seale Rapolai - 15122116

May 2026

1 Dataset

The dataset utilized is the Residential Energy Dataset UK (2014-2020). It contains continuous time-series observations of electricity load demand recorded over a 6-year period. Forecasting electricity consumption is a critical task for energy providers, often addressed through advanced computational methods such as Grammatical Evolution [4]. The original data instances are captured at 15-minute intervals. Given that this is a time-series forecasting problem, past observations must be used to predict future instances. For this experiment, the dataset is restructured through a sliding window by creating lagged attributes; specifically, the models are trained utilizing $m = 8$ contiguous previous values to predict the forthcoming electricity load.

1.1 Preprocessing

The dataset, supplied in CSV format, was loaded into a Python Pandas DataFrame. To maintain temporal integrity, the observations were sorted in ascending order based on the *utc_timestamp* column. *pandas.Timedelta* was then used to verify that the time series was continuous, confirming a uniform 15-minute interval between consecutive data points with no missing entries.
For the feature engineering phase, the *Electricity_load* column was isolated into a new DataFrame to serve as the target variable. To incorporate historical context, the m lagged features were generated by applying shift operations. Specifically, for each $i \in [1, m]$, a new column $load_{t-i}$ was created to represent the electricity load at $t - i$. With m set to 8, these features represent a sliding window of the previous two hours:

- $load_{t-1}$: Load from 15 minutes prior.
- $load_{t-2}$: Load from 30 minutes prior
- Up to $load_{t-8}$ (120 minutes prior).

As a consequence of the shift operations, the initial rows of the dataset contained *null* values where historical data was unavailable. These incomplete rows were removed to ensure a clean dataset for subsequent analysis.

2 Performance metric

The performance metric used to assess the quality of the algorithmic predictor is the **Mean Absolute Percentage Error (MAPE)**.

How it works: MAPE mathematically computes the average absolute percentage difference between the estimated values produced by the decoded phenotype expression tree and the actual known electricity load values in the fitness cases.

Why it was chosen: It is one of the most robust evaluators for numeric regression problems dealing with shifting temporal datasets. Unlike MSE, MAPE provides a scale-invariant metric that prevents the fitness score from being distorted by the naturally fluctuating variance across different seasonal consumption cycles.

3 Standard Grammatical Evolution (GE)

To enforce structural constraints on the evolved predictive models, this assignment utilizes **Standard Grammatical Evolution (GE)**.

GE decouples the evolutionary search space from the physical expression space by maintaining a dual representation:

- **Genotype:** A single flat list of integer codons (typically in the range $[0, 255]$). These codons are used to select expansion rules sequentially during the parsing process.
- **Phenotype:** An executable mathematical expression tree generated by recursively parsing the genotype using a context-free grammar.

Parsing begins at the start symbol of the grammar. At each step, the next non-terminal is expanded by retrieving the next codon in the flat list and choosing the production rule index using a modulo operator:

$$\texttt{rule\_index = codon \% number\_of\_rules\_for\_non\_terminal} \tag{1}$$

If the codon list is exhausted before the derivation terminates, the reading pointer wraps around back to the beginning of the genotype list. A maximum wrap-around limit of 10 wraps (or a maximum node limit of 1000) is enforced to prevent infinite loops. Individuals exceeding these limits are declared invalid and assigned a high penalty fitness value of 10^9.

4 Representation, Terminal Sets, and Function Sets

The solution uses a grammar-defined symbolic representation. The Backus-Naur Form (BNF) grammar maps the flat search space of integer codons to syntax trees, maintaining the mathematical order of operations while ensuring that the resulting equations are transparent and human-interpretable.

The Context-Free Grammar (CFG) used in this assignment is defined below:

$$
\begin{aligned}
\langle \text{root} \rangle &::= \text{StructuredRoot}(\langle \text{expr} \rangle, \langle \text{expr} \rangle) \\
\langle \text{expr} \rangle &::= \text{Function}(\langle \text{op} \rangle, \langle \text{expr} \rangle, \langle \text{expr} \rangle) \mid \text{Variable}(\langle \text{var} \rangle) \mid \text{Constant}(\langle \text{const} \rangle) \\
\langle \text{op} \rangle &::= + \mid - \mid \times \mid \div \mid \max \mid \min \\
\langle \text{var} \rangle &::= \text{load}_{t-1} \mid \text{load}_{t-2} \mid \cdots \mid \text{load}_{t-8} \\
\langle \text{const} \rangle &::= C
\end{aligned}
$$

4.1 Terminal set:

The terminals consist of the $m = 8$ contiguous lagged values extracted from the historical sliding window (i.e., $load_{t-1}$ to $load_{t-8}$), combined with an ephemeral random constant C to inject variance.

- $load_{t-i}$: Electricity load at i previous time steps, for $i \in [1, 8]$.
- C: An ephemeral constant value derived directly from the codon value: $C = \text{round}(((codon\%256)/255.0) * 0.4 - 0.2, 2)$, yielding coefficients cleanly in the range $[-0.2, 0.2]$.

4.2 Function set:

The set consists of basic arithmetic primitives and threshold operators configured as binary functions (Arity = 2):

- Arithmetic operators: Addition (+), Subtraction (−), Multiplication (×), and safe Division (÷, which returns 1 when the denominator is zero).
- Threshold operators: Minimum (min) and Maximum (max), which capture peak energy loads and seasonal demand limits.

Equation 2 below shows an example of a decoded phenotype formula:

$$\hat{y}_t = \text{StructuredRoot}(\max(load_{t-4}, load_{t-7}) + \min((load_{t-4} - load_{t-8}), load_{t-3})) \quad (2)$$

Figure 1 visualizes the phenotype structure corresponding to Equation 2.

5 Initial population generation

The initial population is generated using **Sensible Initialisation** in association with the Ramped Half-and-Half method [3] to guarantee valid initial genotypes.

For a given population size, the derivation process employs a 50/50 split between two creation strategies:

- **Full method:** During the recursive grammar derivation, if the current depth is strictly less than the target depth, the expansion rule is locked to index 0 (`Function`). This forces the tree to grow until it reaches the maximum depth limit, where it terminates.

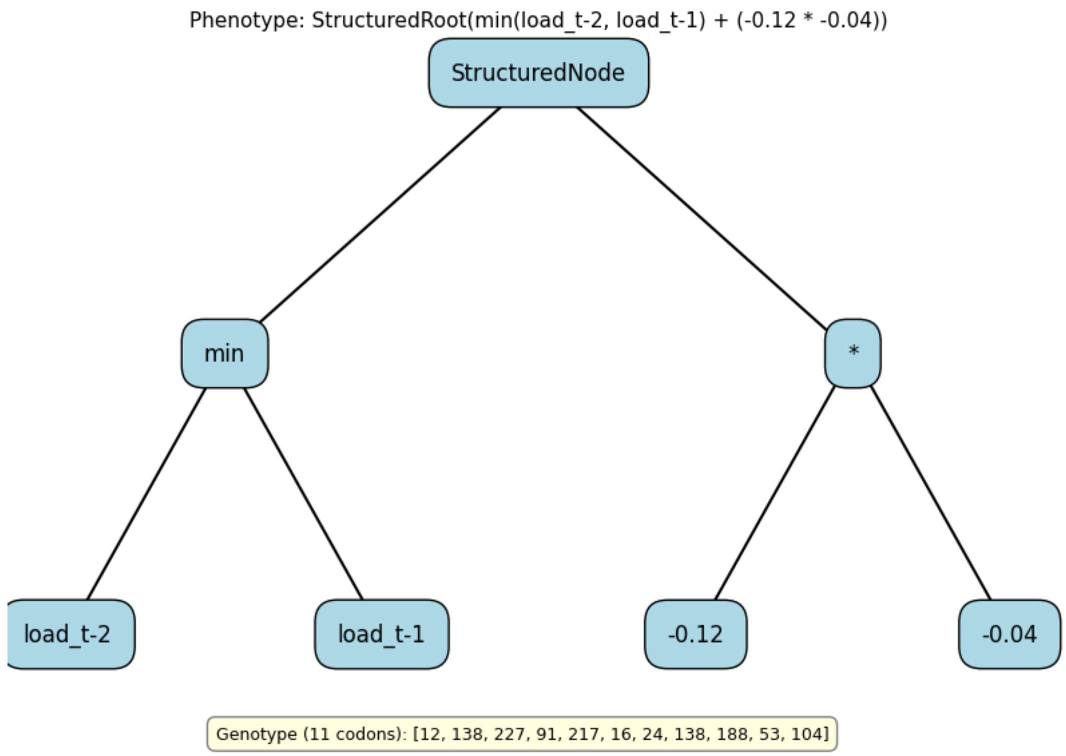

Figure 1: Example phenotype tree representing a grammar-derived individual.

- **Grow method:** While the current depth is less than the target depth, the rule index is selected completely at random from all valid production rules (`Function`, `Variable`, or `Constant`), allowing branches to terminate early and produce asymmetrical trees.

Once the target depth is reached, rule selection is restricted to terminal options (Variable or Constant) to force termination. To map these derivations to the flat list representation, the selected rule indices are mapped to codon values (`codon % len(rules) == r_idx`), generating a standard flat integer array of standard 8-bit integers ($[0, 255]$).

6 Fitness

6.1 Fitness Cases

The algorithm evaluates individuals across a dynamic sliding window extracted from the time-series dataset. A batch window (representing a specific number of days, governed by the `window-size` argument) is selected. Each row within this temporal slice acts as a single fitness case, offering the 8 lagged input variables ($load_{t-1}$ to $load_{t-8}$) and the target `Electricity_load`. The window shifts forward every generation, ensuring the population learns to generalize across seasonal consumption shifts instead of overfitting to a static dataset.

6.2 Fitness Evaluation

During evaluation, each individual genotype is decoded into its phenotype expression tree. The raw fitness is computed using the scale-invariant **Mean Absolute Percentage Error (MAPE)**.

The exact fitness function $F(T)$ for a decoded phenotype tree T evaluated over N fitness cases is defined in Equation 3:

$$F(T) = \frac{1}{N} \sum_{i=1}^{N} \begin{cases} 10 & \text{if } T(x_i) \text{ results in an exception} \\ \left| \frac{y_i - T(x_i)}{\max(|y_i|, 10^{-8})} \right| & \text{otherwise} \end{cases} \tag{3}$$

where N is the number of fitness cases, x_i is the lagged vector, $T(x_i)$ is the predicted output, and y_i is the target load.

If the individual failed to decode (e.g., wrap-around limits exceeded), a high penalty of 10^9 is returned to filter it out. Any evaluation triggering mathematical exceptions (e.g., division by zero) returns a penalty of $+10$. Since this is an error metric, the evolution loop actively minimizes $F(T)$ toward zero.

7 selection method

The algorithm utilizes **Tournament Selection** [3] to select individuals for genetic operators. For each selection event, the algorithm samples a uniform subset of individuals from the population (defined by the `tournament-size` parameter). These candidates compete, and the individual with the lowest phenotype MAPE wins the tournament and is passed to the genetic operators.

Tournament selection provides several key advantages:

1. **Scale Independence:** By comparing only the relative rank within the tournament, the selection is immune to large fluctuations in absolute MAPE values across shifting seasonal windows.

2. **Adjustable Selection Pressure:** Tuning the tournament size alters the convergence velocity. A small size (e.g., 2) maintains genetic diversity by allowing weaker genotypes to occasionally breed, whereas a larger size drives rapid convergence.

3. **High Efficiency:** Selecting parents operates in $O(k)$ time (where k is the tournament size), bypassing the computational overhead of global sorting.

8 Genetic operators

The standard GE algorithm applies three core genetic operators at the genotype level to navigate the flat search space: **Crossover**, **Mutation**, and **Reproduction**. These operators are applied based on strict proportional rates.

1. **Reproduction (Elitism):** The top percentage of the population (governed by the reproduction rate) is identified by sorting the fitness values. The genotype lists of these elite individuals are copied directly to the next generation without modification, preserving the absolute best mathematical equations.

2. **Flat Genotype Crossover:** Two parents are chosen via Tournament Selection. A random single-point cut is chosen within their overlapping length on the flat integer lists, and the codon lists are swapped to generate two child genotypes. The child genotypes are decoded, and if their phenotype depths respect the global `max_depth` constraint, they are accepted; otherwise, copies of the parents are returned to curb uncontrolled bloat.

3. **Flat Genotype Mutation:** A single parent is selected. Codon flip mutation is applied to its flat list: every codon has a 15% chance of being replaced by a completely new random integer in $[0, 255]$. Constant tuning is achieved organically, as mutating a constant codon directly shifts its derived floating-point coefficient. If the mutated individual violates the max depth constraint or fails to decode, the mutation retries up to 3 times before falling back to the original parent.

8.1 Structural Similarity in Crossover

To prevent redundant crossover events that lead to local optima, the system implements morphological similarity tracking using two distinct indices computed on the decoded phenotype trees [1]:

- **Global Index (GI):** Measures the number of matching function nodes shared between two phenotypes from the root down to a cutoff depth limit (half of the maximum depth).

- **Local Index (LI):** Counts matching function and terminal nodes in the subtrees beyond the cutoff depth.

9 Experimental setup

Hardware specifications: Intel(R) Core(TM) i7-8750H CPU @ 2.20GHz with 16 GB of RAM.

Table 1 lists the parameters applied during this simulation:

The parameters balance computational efficiency, search space traversal, and phenotypic interpretability:

- **Population Size (100) & Max Depth (5):** Prevents tree bloat and ensures the evolved equations are lean, highly interpretable, and generalized.

- **Genetic Rates (80% Crossover / 19% Mutation / 1% Elitism):** Maximizes constructive recombination while continuously introducing structural and constant-level variance.

- **Tournament Size (2):** Maintains diverse selection dynamics, allowing suboptimal but structurally novel individuals to breed.

- **Generations (70):** Explicitly calculated to cover the entire historical dataset (201,596 instances) over the sliding window length of 30 days (2,880 instances per generation):

$$\text{Generations} = \left\lceil \frac{\text{Total Instances}}{\text{Instances per Window}} \right\rceil = \left\lceil \frac{201,596}{2,880} \right\rceil \approx 70 \tag{4}$$

Parameter	Value
Population size	100
Max depth	5
Initial population generation	Sensible Ramped half-and-half
Fitness function	$MeanAbsolutePercentageError(MAPE)$
Test Cases / Window Constraints	30 days sliding window chunk offsets
Selection method	Tournament Selection
Tournament Size	2
Mutation rate	19% (19 individuals per 100)
Crossover rate	80% (80 individuals per 100)
Reproduction Rate	1% (1 individual per 100)
Number of generations	70
Termination criteria	Fixed generation limit reached

Table 1: Standard GE configuration parameters

10 Results

The Standard GE algorithm successfully converged across all experimental runs. The system demonstrated strong generalization capacities, navigating the tumbling sliding window that exposes the population to new temporal offsets every generation.

Run NO.	Random seed	Best Individual (MAPE)
1	30	0.1889
2	45	0.1922
3	83	0.1924
4	51	0.1887
5	48	0.1872
6	84	0.1898
7	1	0.1887
8	20	0.1889
9	70	0.1918
10	12	0.2015
mean		0.1910
Std. Div		0.0041

Table 2: Execution results for the best 10 runs

As shown in Table 2, across 10 independent experimental runs, the best individual consistently achieved a MAPE between **18.7%** and **20.1%**. The mean across all runs is **19.10%**, with a narrow standard deviation of **0.41%**, highlighting the algorithm's robustness to random seed initializations.

The best individual across all runs was found in **Run 5 (Seed 48)**, with a MAPE of **18.72%**. The evolved phenotype heavily utilizes recursive minimum and maximum structures to model peak loads. Figure 2 illustrates this tree.

Due to the online sliding window, the algorithm experiences minor dataset shock at the start of each generation. However, the average and worst errors converge sharply as

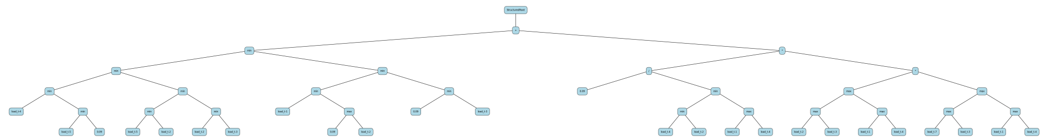

Figure 2: Binary Expression Tree of the best individual from run 5, using 48 as random seed.

shown in Figure 3, crashing downward from extreme generation-0 outliers and squeezing tightly against the elite baseline.

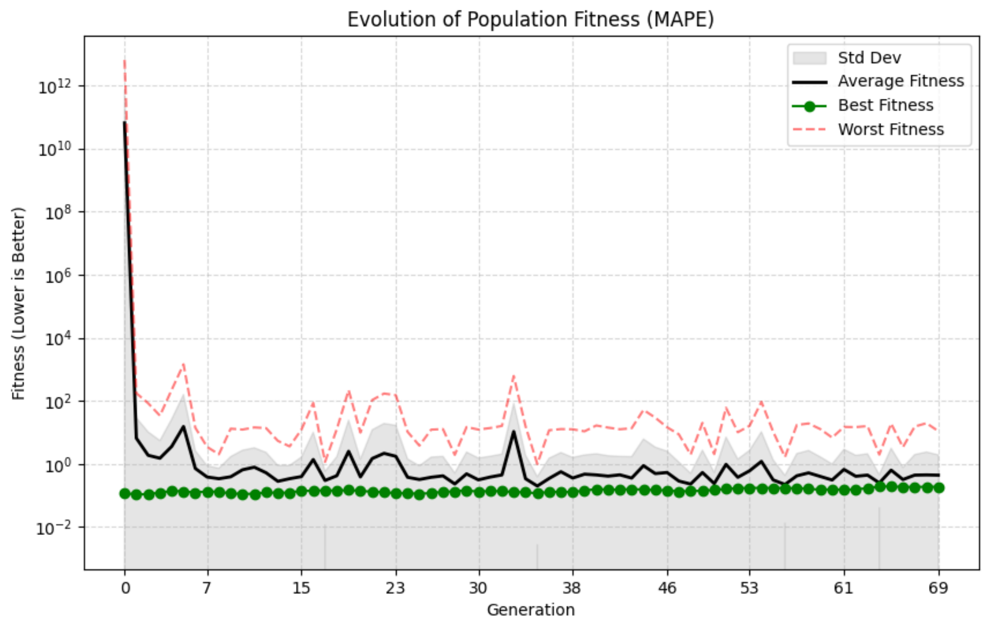

Figure 3: Evolution of population fitness over generations.

Figure 4 exhibits minor fluctuations (bouncing between roughly 12% and 20% MAPE), which is an expected consequence of the tumbling sliding window. Every generation, the elite tree is evaluated on unseen data, reflecting shifting seasonal predictability. The stable elite MAPE confirms the discovery of highly generalized models that resist overfitting.

11 Run times

To ensure computational scalability across the 5.7 years of temporal data, the fitness evaluation pipeline was fully refactored to utilize asynchronous multiprocessing, distributing the load of evaluating 100 individuals across all available CPU cores.

Across the 10 experimental runs, processing 70 generations took between **967.21 seconds** (16.1 minutes) and **1,402.03 seconds** (23.3 minutes). On average, a complete traversal of the 201,596-row dataset took roughly **20.5 minutes**.

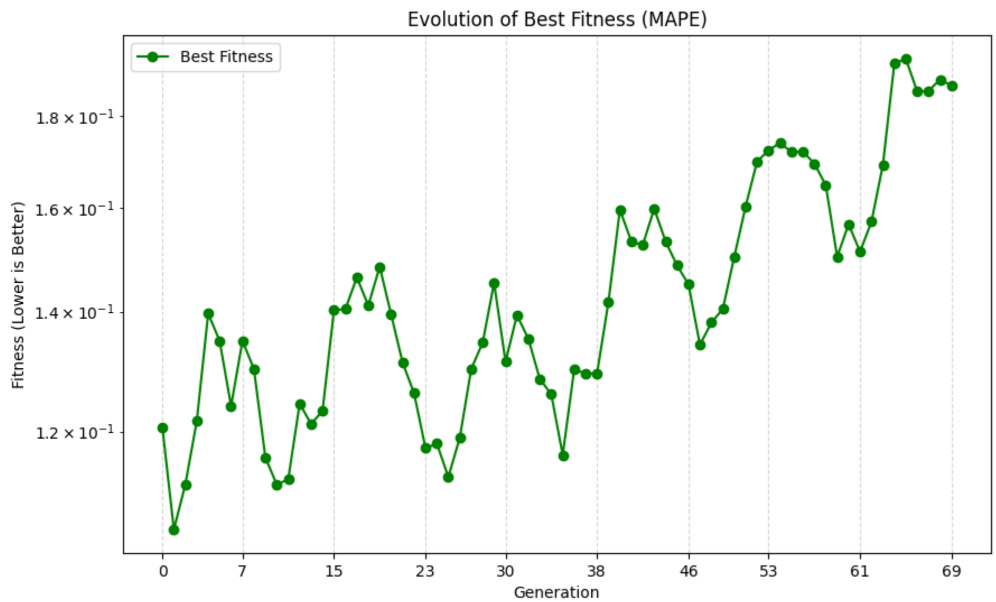

Figure 4: Evolution of best individual's fitness per generation.

The run times shown in Table 3 prove that the algorithm successfully balances structural complexity with processing speed, maintaining a relatively lean tree architecture (Max Depth = 5) while computing tens of millions of mathematical operations across the feature set. The overall mean execution time of **1234.75 seconds** (approximately 20.5 minutes) demonstrates highly efficient traversal of the 5.7-year dataset. The standard deviation of **184.47 seconds** (roughly ± 3 minutes) indicates that while the computational load is highly stable, it naturally fluctuates depending on the specific random seed. This variance is expected in Genetic Programming, as seeds that organically evolve structurally denser, mathematically heavy equations early in the simulation will require more CPU cycles to process across the dataset than seeds that converge upon leaner, shallower equations.

12 Comparison of Previous Assignments

The fundamental evolution from Assignment 1 and 2 to Assignment 3 lies in the mechanism of structural enforcement: the transition from Genetic Programming (GP) to Standard Grammatical Evolution (GE). This architectural upgrade completely redefines how structural validity is maintained during the evolutionary process.

12.1 Context-Free Grammar as the Structural Core

In standard GP (Assignment 1) and Structure-Based GP (Assignment 2), the search space and the expression space were identical: operations were performed directly on the morphological trees. While Assignment 2 utilized a similarity index to manage structural diversity, it still relied on tree-based crossover which inherently risked creating invalid or bloated mathematical expressions if not strictly constrained.

Run NO.	Execution Time (Seconds)
1	1370.14
2	1350.28
3	1369.35
4	1379.87
5	1378.09
6	1402.03
7	1088.77
8	1056.39
9	985.40
10	967.21
mean	1234.75
Std. Div	184.47

Table 3: Execution results for the best 10 runs

In contrast, Assignment 3 introduces the **Context-Free Grammar (CFG) and Genotype-Phenotype mapping** as its primary structural component. By decoupling the flat integer search space (genotype) from the executable trees (phenotype), GE offloads all structural responsibilities to the grammar.

12.2 Enforcing Structural Validity and Constraints

The grammar serves as an absolute structural blueprint. It explicitly enforces the order of operations, prevents mathematical type mismatches, and restricts the search space exclusively to valid equations. Because crossover and mutation occur on the flat integer lists, the genetic operators are completely agnostic to the phenotype structure.

However, the integrity of the resulting phenotype is mathematically guaranteed by the CFG during the decoding phase. This strict, decoupled structure mitigates the destructive crossover events seen in Assignment 1 and eliminates the need for expensive morphological similarity calculations seen in Assignment 2, all while preserving the mathematical interpretability of the evolved models.

References

[1] Kapoor, R., & Pillay, N. (2024). A genetic programming approach to the automated design of CNN models for image classification and video shorts creation. *Genetic Programming and Evolvable Machines*, 25(10). https://doi.org/10.1007/s10710-024-09483-5

[2] Montana, D. J. (1995). Strongly Typed Genetic Programming. *Evolutionary Computation*, 3(2), 199–230.

[3] Poli, R., Langdon, W. B., & McPhee, N. F. (2008). *A Field Guide to Genetic Programming*. Lulu.com.

[4] Castelli, M., Vanneschi, L., & Silva, S. (2015). Prediction of electricity consumption by using genetic programming with semantic-based genetic operators. *Statistical Analysis and Data Mining*, 8(4), 242–256.